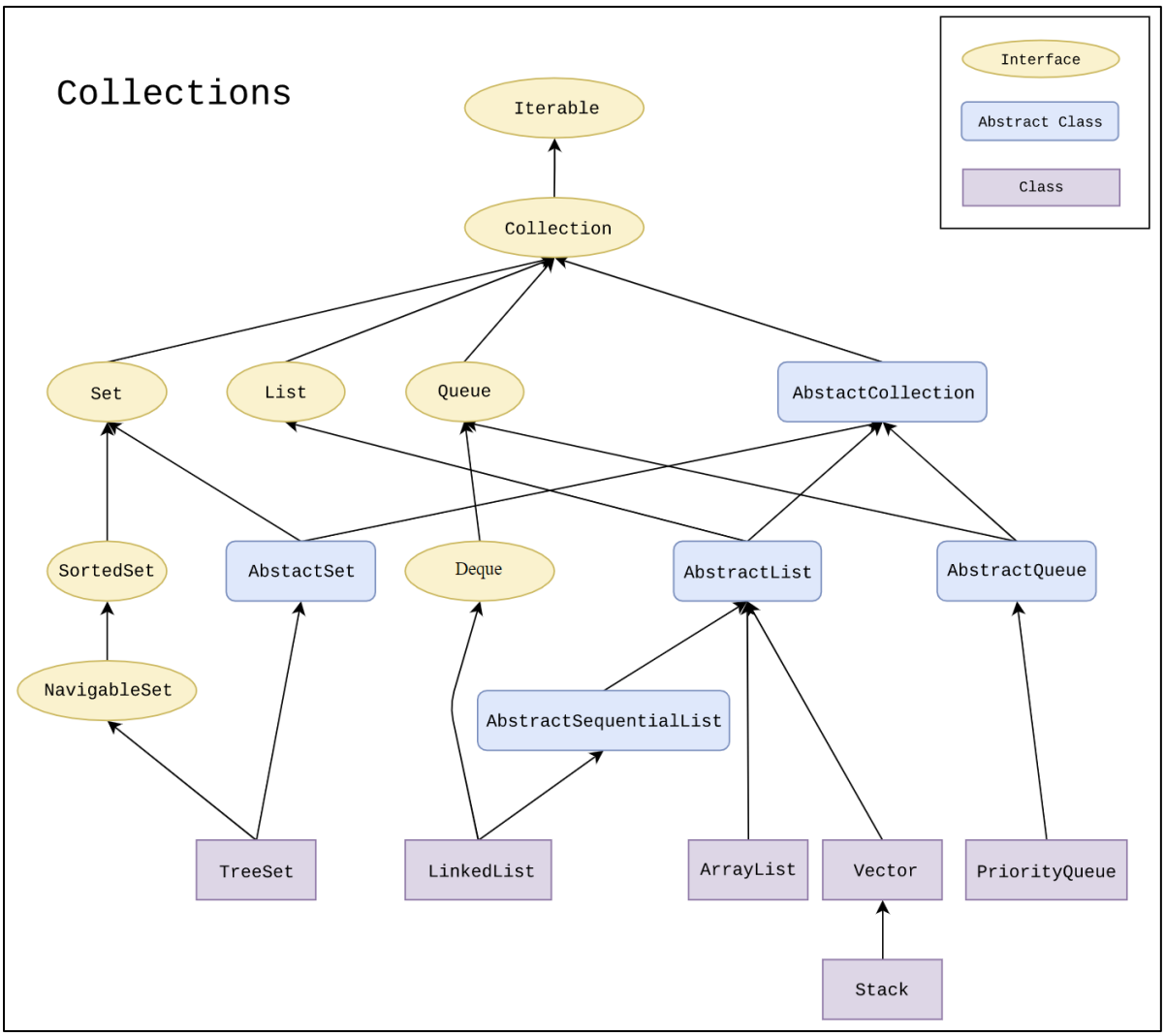
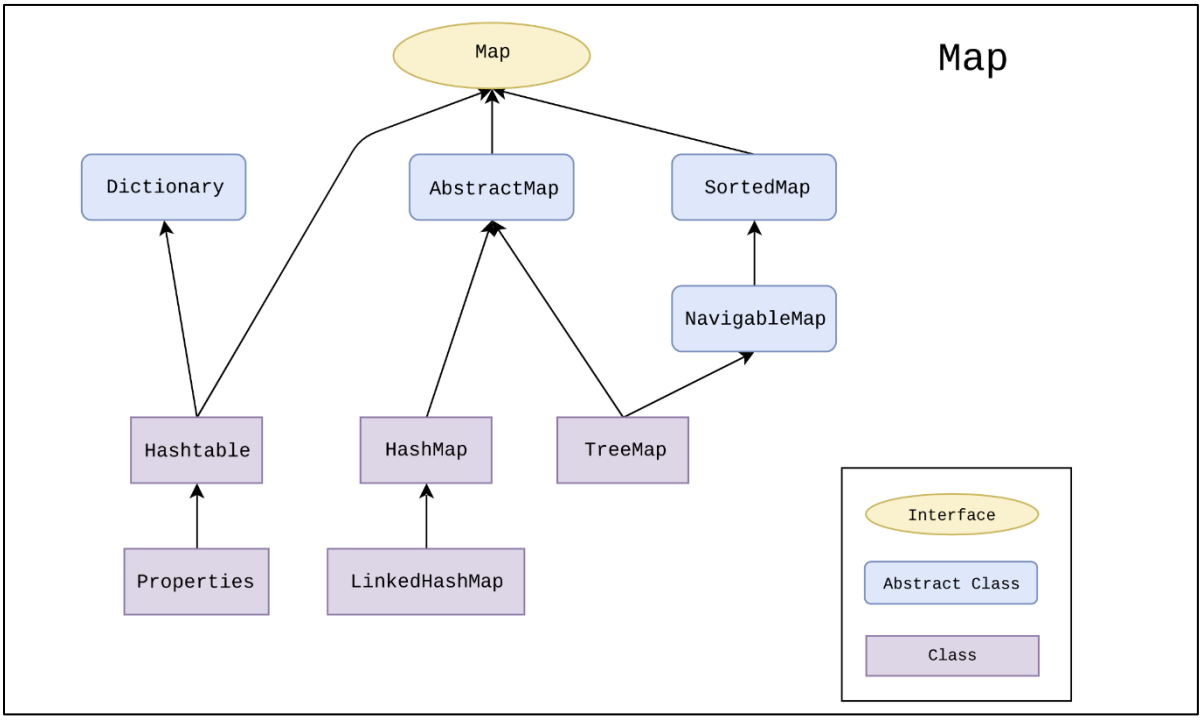
PKG 12: QUEUE, DEQUE, PRIORITY QUEUE





QUEUE

```java
import java.util.LinkedList;
import java.util.Queue; // Queue: add, element, offer, peek, poll, remove

public class QueueDemo {

    public static void main(String[] args) {
        Queue<Integer> myQueue = new LinkedList<>();

        // add() isEmpty() peek()
        System.out.println("Elements (FRONT): " + myQueue);
        System.out.println("Is Empty: " + myQueue.isEmpty());       // Collection
        System.out.println("Head: " + myQueue.peek());
        // System.out.println("Head: " + myQueue.element());        // Error if empty
        for (int i = 0; i < 7; i++) {
            myQueue.add(i);                                         // vs. offer()
        }

        // contains() size() peek()
        System.out.println("\nElements (FRONT): " + myQueue);
        System.out.println("Size: " + myQueue.size());
        System.out.println("Is Empty: " + myQueue.isEmpty());
        System.out.println("Head, Peek: " + myQueue.peek());
        System.out.println("Head, Element: " + myQueue.element());   // vs. peek()

        // Collection: contains() remove()
        System.out.println("\nContains: " + myQueue.contains(1));
        System.out.println("Contains: " + myQueue.contains(10));
        System.out.println("Removed: " + myQueue.remove(3));         // Collection

        // remove() remove(entry) poll() clear()
        System.out.println("\nElements (FRONT): " + myQueue);
        System.out.println("Removed: " + myQueue.remove());
        System.out.println("Removed: " + myQueue.poll());
        System.out.println("Elements (FRONT): " + myQueue);
        myQueue.clear();
        System.out.println("\nSize: " + myQueue.size());
        System.out.println("Is Empty: " + myQueue.isEmpty());        // Collection
        System.out.println("Elements (FRONT): " + myQueue);
        //System.out.println("Removed: " + myQueue.remove());        // Error if empty
        System.out.println("Remove, Poll: " + myQueue.poll());       // vs. remove()
    }
}
```

```
Elements (FRONT): []
Is Empty: true
Head: null

Elements (FRONT): [0, 1, 2, 3, 4, 5,
6]
Size: 7
Is Empty: false
Head, Peek: 0
Head, Element: 0

Contains: true
Contains: false
Removed: true

Elements (FRONT): [0, 1, 2, 4, 5, 6]
Removed: 0
Removed: 1
Elements (FRONT): [2, 4, 5, 6]

Size: 0
Is Empty: true
Elements (FRONT): []
Remove, Poll: null
```

java.util.Collection: addAll, clear, contains, containsAll, equals, hashCode, isEmpty, iterator, parallelStream, remove, removeAll, removeIf, retainAll, size, spliterator, stream, toArray, toArray, toArray

java.lang.Iterable:forEach

DEQUE

java.util.Collection: clear, containsAll, equals, hashCode, isEmpty, parallelStream, removeAll, removeIf, retainAll, spliterator, stream, toArray, toArray, toArray

java.lang.Iterable: forEach

```
import java.util.Deque;
import java.util.LinkedList;

public class DequeDemo {
    public static void main(String[] args) {
        Deque myDeck = new LinkedList<>();

        // PEEK
        System.out.println("PEEK");
        // System.out.println(myDeck.element());  // Error
        // System.out.println(myDeck.getFirst()); // Error
        // System.out.println(myDeck.getLast());  // Error
        System.out.println(myDeck.peek());        // null
        System.out.println(myDeck.peekFirst());   // null
        System.out.println(myDeck.peekLast());    // null

        // POP
        System.out.println("\nPOP");
        // System.out.println(myDeck.pop());         // Error
        // System.out.println(myDeck.remove());      // Error
        // System.out.println(myDeck.removeFirst()); // Error
        // System.out.println(myDeck.removeLast());  // Error
        System.out.println(myDeck.poll());           // null
        System.out.println(myDeck.pollFirst());      // null
        System.out.println(myDeck.pollLast());       // null

        // ADD
        System.out.println("\nADD");
        myDeck.add("E1-T");        // [E1-T]                                  // boolean
        myDeck.addFirst("E2-H");   // [E2-H, E1-T]
        myDeck.addLast("E3-T");    // [E2-H, E1-T, E3-T]
        myDeck.push("E4-H");       // [E4-H, E2-H, E1-T, E3-T]
        myDeck.offer("E5-T");      // [E4-H, E2-H, E1-T, E3-T, E5-T]          // boolean
        myDeck.offerFirst("E6-H"); // [E6-H, E4-H, E2-H, E1-T, E3-T, E5-T]
        myDeck.offerLast("E7-T");  // [E6-H, E4-H, E2-H, E1-T, E3-T, E5-T, E7-T]
        System.out.println(myDeck);// [E6-H, E4-H, E2-H, E1-T, E3-T, E5-T, E7-T]

        // PEEK
        System.out.println("\nPEEK");
        System.out.println(myDeck.element());     // E6-H
        System.out.println(myDeck.getFirst());    // E6-H
        System.out.println(myDeck.getLast());     // E7-T
        System.out.println(myDeck.peek());        // E6-H
        System.out.println(myDeck.peekFirst());   // E6-H
        System.out.println(myDeck.peekLast());    // E7-T

        // POP
        System.out.println("\nPOP");
        System.out.println(myDeck);// [E6-H, E4-H, E2-H, E1-T, E3-T, E5-T, E7-T]
        System.out.println(myDeck.pop());         // E6-H
        System.out.println(myDeck.remove());      // E4-H
        System.out.println(myDeck.removeFirst()); // E2-H
        System.out.println(myDeck.removeLast());  // E7-T
        System.out.println(myDeck.poll());        // E1-T
        System.out.println(myDeck.pollFirst());   // E3-T
        System.out.println(myDeck.pollLast());    // E5-T
```

```java
        // MORE
        myDeck.add("A");
        myDeck.add("B");
        myDeck.add("A");
        myDeck.add("B");
        myDeck.add("A");
        System.out.println(myDeck);          // [A, B, A, B, A]
        myDeck.remove("B");                  // [A, A, B, A]      // boolean
        myDeck.removeFirstOccurrence("A"); // [A, B, A]         // boolean
        myDeck.removeLastOccurrence("B");  // [A, A]            // boolean
        System.out.println(myDeck.contains("C")); // false
        System.out.println(myDeck.contains("A")); // true
        System.out.println(myDeck.size());     // 2
        System.out.println(myDeck.isEmpty()); // false          // Collection
        myDeck.clear();                                         // Collection
        System.out.println(myDeck.isEmpty()); // true           // Collection
    }
}
```

PRIORITY QUEUE

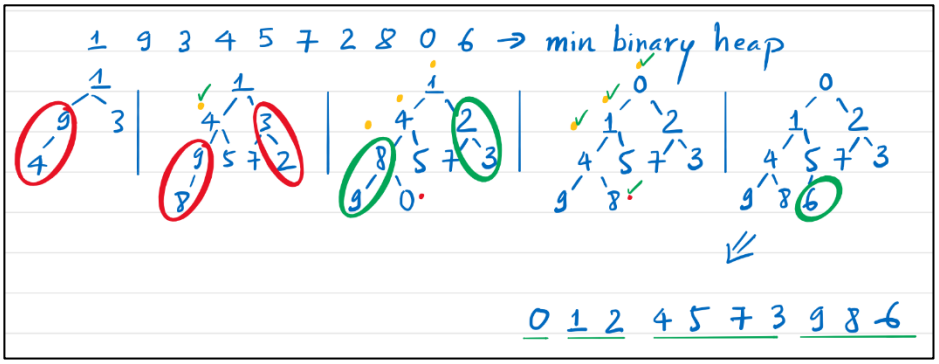
```java
import java.util.PriorityQueue;

public class PriorityQueueDemo1 {

    public static void main(String args[]) {
        PriorityQueue<Integer> myPQ = new PriorityQueue<>();

        // Collection isEmpty()
        System.out.println(myPQ.isEmpty()); // true

        // add() offer()
        myPQ.add(1);
        myPQ.offer(9);
        myPQ.add(3);
        myPQ.offer(4);
        myPQ.add(5);
        myPQ.add(7);
        myPQ.add(2);
        myPQ.add(8);
        myPQ.offer(0);
        myPQ.offer(6);
```




```java
        // display 1, PQ is stored in a Binary Heap
        System.out.println(myPQ); // [0, 1, 2, 4, 5, 7, 3, 9, 8, 6]

        // contains() peek() remove()
        System.out.println(myPQ.contains(6)); // true
        System.out.println(myPQ.peek());      // 0
        System.out.println(myPQ.remove());    // 0          // Collection
        System.out.println(myPQ.remove(9));   // true

        // size()
        int originalSize = myPQ.size();

        // display 2, highest priority
        for (int i = 0; i < originalSize; i++) {
            System.out.print(myPQ.poll() + " ");
        } // 1 2 3 4 5 6 7 8
    }
}
```

```
/* OUTPUT
true
[0, 1, 2, 4, 5, 7, 3, 9, 8, 6]
true
0
0
true
1 2 3 4 5 6 7 8
*/
```

```java
import java.util.PriorityQueue;
import java.util.Queue;

class PC implements Comparable<PC> {
    private final int cpuSpeed;
    private final String name;

    public PC(int cpuSpeed, String name) {
        this.cpuSpeed = cpuSpeed;
        this.name = name;
    }

    public int getCpuSpeed() {
        return this.cpuSpeed;
    }

    public String getName() {
        return this.name;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof PC)) {
            return false;
        }
        PC p = (PC) o;
        return this.cpuSpeed == p.getCpuSpeed();
    }

    @Override
    public int hashCode() {
        return this.cpuSpeed;
    }

    @Override
    public String toString() {
        return this.cpuSpeed + ", " + this.name;
    }

    @Override
    public int compareTo(PC pc) {

        int tmpSpeed = pc.getCpuSpeed();
        String pcName = pc.getName();

        if (this.getCpuSpeed() < tmpSpeed) {
            return -1;
        }

        if (this.getCpuSpeed() > tmpSpeed) {
            return 1;
        }

        if (this.getCpuSpeed() == tmpSpeed) {
            return this.getName().compareTo(pcName);
        }

        // Wrong types compared
        return 0;
    }
}
```

```java
public class PriorityQueueDemo2 {

    public static void main(String[] args) {
        Queue<PC> pq = new PriorityQueue<>();
        pq.add(new PC(1, "Dell D"));
        pq.add(new PC(4, "Sony S"));
        pq.add(new PC(2, "Apple A"));
        pq.add(new PC(3, "HP H"));
        pq.add(new PC(4, "Google G"));
        pq.add(new PC(4, "Google O"));
        pq.add(new PC(4, "Google O"));
        pq.add(new PC(4, "Google G"));
        pq.add(new PC(4, "Google L"));
        pq.add(new PC(4, "Google E"));

        while (pq.peek() != null) {
            System.out.println(pq.poll());
        }
    }
}
```

```
1, Dell D
2, Apple A
3, HP H
4, Google E
4, Google G
4, Google G
4, Google L
4, Google O
4, Google O
4, Sony S
```

java.util.AbstractQueue
addAll, element, remove

java.util.AbstractCollection
containsAll, isEmpty, toString

java.lang.Object
clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

java.util.Collection
containsAll, equals, hashCode, isEmpty, parallelStream, size, stream, toArray

java.util.Queue
peek, poll

JAVA COLLECTIONS FRAMEWORK

From Wikipedia, the free encyclopedia

The Java collections framework is a set of classes and interfaces that implement commonly reusable collection data structures.
Although referred to as a framework, it works in a manner of a library. The collections framework provides both interfaces that define various collections and classes that implement them.

Differences to Arrays

Collections and arrays are similar in that they both hold references to objects, and they can be managed as a group. However, unlike arrays, collections do not need to be assigned a certain capacity when instantiated. Collections can also grow and shrink in size automatically when objects are added or removed. Collections cannot hold basic data type elements (primitive types) such as int, long, or double; instead, they hold Wrapper Classes such as Integer, Long, or Double.

History

Collection implementations in pre-JDK 1.2 versions of the Java platform included few data structure classes, but did not contain a collections framework. The standard methods for grouping Java objects were via the array, the Vector, and the Hashtable classes, which unfortunately were not easy to extend, and did not implement a standard member interface.
To address the need for reusable collection data structures, several independent frameworks were developed, the most used being Doug Lea's Collections package, and ObjectSpace Generic Collection Library (JGL), whose main goal was consistency with the C++ Standard Template Library (STL).
The collections framework was designed and developed primarily by Joshua Bloch, and was introduced in JDK 1.2. It reused many ideas and classes from Doug Lea's Collections package, which was deprecated as a result. Sun Microsystems chose not to use the ideas of JGL, because they wanted a compact framework, and consistency with C++ was not one of their goals.
Doug Lea later developed a concurrency package, comprising new Collection-related classes. An updated version of these concurrency utilities was included in JDK 5.0 as of JSR 166.

Architecture

Almost all collections in Java are derived from the java.util.Collection interface. Collection defines the basic parts of all collections. The interface states the add() and remove() methods for adding to and removing from a collection respectively. Also required is the toArray() method, which converts the collection into a simple array of all the elements in the collection. Finally, the contains() method checks if a specified element is in the collection. The Collection interface is a subinterface of java.lang.Iterable, so any Collection may be the target of a for-each statement. (The Iterable interface provides the iterator() method used by for-each statements.) All collections have an iterator that goes through all of the elements in the collection. Additionally, Collection is a generic. Any collection can be written to store any class. For example, Collection<String> can hold strings, and the elements from the collection can be used as strings without any casting required. Note that the angled brackets < > can hold a type argument that specifies which type the collection holds.

Three Types of Collection

There are three generic types of collection: ordered lists, dictionaries/maps, and sets.
Ordered lists allows the programmer to insert items in a certain order and retrieve those items in the same order. An example is a waiting list. The base interfaces for ordered lists are called List and Queue.

Dictionaries/Maps store references to objects with a lookup key to access the object's values. One example of a key is an identification card. The base interface for dictionaries/maps is called Map.
Sets are unordered collections that can be iterated and contain each element at most once. The base interface for sets is called Set.

List interface

Lists are implemented in the collections framework via the java.util.List interface. It defines a list as essentially a more flexible version of an array. Elements have a specific order, and duplicate elements are allowed. Elements can be placed in a specific position. They can also be searched for within the list. Two examples for concrete classes that implement List are:

- java.util.ArrayList, which implements the list as an array. Whenever functions specific to a list are required, the class moves the elements around within the array in order to do it.
- java.util.LinkedList. This class stores the elements in nodes that each have a pointer to the previous and next nodes in the list. The list can be traversed by following the pointers, and elements can be added or removed simply by changing the pointers around to place the node in its proper place.

Stack class

Stacks are created using java.util.Stack. The stack offers methods to put a new object on the stack (method push()) and to get objects from the stack (method pop()). A stack returns the object according to last-in-first-out (LIFO), e.g. the object which was placed latest on the stack is returned first. java.util.Stack is a standard implementation of a stack provided by Java. The Stack class represents a last-in-first-out (LIFO) stack of objects. It extends class java.util.Vector with five operations that allow a vector to be treated as a stack. The usual push and pop operations are provided, as well as a method to peek at the top item on the stack, a method to test for whether the stack is empty, and a method to search the stack for an item and discover how far it is from the top. When a stack is first created, it contains no items.

Queue interfaces

The java.util.Queue interface defines the queue data structure, which stores elements in the order in which they are inserted. New additions go to the end of the line, and elements are removed from the front. It creates a first-in first-out system. This interface is implemented by java.util.LinkedList, java.util.ArrayDeque, and java.util.PriorityQueue. LinkedList, of course, also implements the List interface and can also be used as one. But it also has the Queue methods. ArrayDeque implements the queue as an array. Both LinkedList and ArrayDeque also implement the java.util.Deque interface, giving it more flexibility.

java.util.Queue can be used more flexibly with its subinterface, java.util.concurrent.BlockingQueue. The BlockingQueue interface works like a regular queue, but additions to and removals from the queue are blocking. If remove is called on an empty queue, it can be set to wait either a specified time or indefinitely for an item to appear in the queue. Similarly, adding an item is subject to an optional capacity restriction on the queue, and the method can wait for space to become available in the queue before returning.

java.util.PriorityQueue implements java.util.Queue, but also alters it. Instead of elements being ordered in the order in which they are inserted, they are ordered by priority. The method used to determine priority is either the compareTo() method in the elements or a method given in the constructor. The class creates this by using a heap to keep the items sorted.

Double-ended queue (deque) interfaces

The java.util.Queue interface is expanded by the java.util.Deque subinterface. Deque creates a double-ended queue. While a regular queue only allows insertions at the rear and removals at the front, the deque allows insertions or removals to take place both at the front and the back. A deque is like a queue that can be used forwards or backwards, or both at once. Additionally, both a forwards and a backwards iterator can be generated. The Deque interface is implemented by java.util.ArrayDeque and java.util.LinkedList.

The java.util.concurrent.BlockingDeque interface works similarly to java.util.concurrent.BlockingQueue. The same methods for insertion and removal with time limits for waiting for the insertion or removal to become possible are provided. However, the interface also provides the flexibility of a deque. Insertions and removals can take place at both ends. The blocking function is combined with the deque function.

Set interfaces

Java's java.util.Set interface defines the set. A set can't have any duplicate elements in it. Additionally, the set has no set order. As such, elements can't be found by index. Set is implemented by java.util.HashSet, java.util.LinkedHashSet, and java.util.TreeSet. HashSet uses a hash table. More specifically, it uses a java.util.HashMap to store the hashes and elements and to prevent duplicates. java.util.LinkedHashSet extends this by creating a doubly linked list that links all of the elements by their insertion order. This ensures that the iteration order over the set is predictable. java.util.TreeSet uses a red-black tree implemented by a java.util.TreeMap. The red-black tree makes sure that there are no duplicates. Additionally, it allows TreeSet to implement java.util.SortedSet.

The java.util.Set interface is extended by the java.util.SortedSet interface. Unlike a regular set, the elements in a sorted set are sorted, either by the element's compareTo() method, or a method provided to the constructor of the sorted set. The first and last elements of the sorted set can be retrieved, and subsets can be created via minimum and maximum values, as well as beginning or ending at the beginning or ending of the sorted set. The SortedSet interface is implemented by java.util.TreeSet.

java.util.SortedSet is extended further via the java.util.NavigableSet interface. It's similar to SortedSet, but there are a few additional methods. The floor(), ceiling(), lower(), and higher() methods find an element in the set that's close to the parameter. Additionally, a descending iterator over the items in the set is provided. As with SortedSet, java.util.TreeSet implements NavigableSet.

Map interfaces

Maps are defined by the java.util.Map interface in Java. Maps are simple data structures that associate a key with an element. This lets the map be very flexible. If the key is the hash code of the element, the map is essentially a set. If it's just an increasing number, it becomes a list. Maps are implemented by java.util.HashMap, java.util.LinkedHashMap, and java.util.TreeMap. HashMap uses a hash table. The hashes of the keys are used to find the elements in various buckets. LinkedHashMap extends this by creating a doubly linked list between the elements, allowing them to be accessed in the order in which they were inserted into the map. TreeMap, in contrast to HashMap and LinkedHashMap, uses a red-black tree. The keys are used as the values for the nodes in the tree, and the nodes point to the elements in the map.

The java.util.Map interface is extended by its subinterface, java.util.SortedMap. This interface defines a map that's sorted by the keys provided. Using, once again, the compareTo() method or a method provided in the constructor to the sorted map, the key-element pairs are sorted by the keys. The first and last keys in the map can be called. Additionally, submaps can be created from minimum and maximum keys. SortedMap is implemented by java.util.TreeMap.

The java.util.NavigableMap interface extends java.util.SortedMap in various ways. Methods can be called that find the key or map entry that's closest to the given key in either direction. The map can also be reversed, and an iterator in reverse order can be generated from it. It's implemented by java.util.TreeMap.